

THE MATCOM MINIX

Informe Técnico

Miguel Zamora Grupo: C-212

Alfonso Tejeda Rodríguez Grupo: C-212

May 8, 2026

Contents

1	Introducción	2
2	Desarrollo y resultados por componente	2
2.1	Instalación y configuración de VirtualBox y MINIX	2
2.2	Personalización del mensaje de bienvenida	3
2.3	Depuración de un bug en <code>pthread</code>	4
2.4	Implementación del comando <code>tree</code>	5
2.5	Penalización por uso intensivo de CPU	8
3	Resultados globales y discusión	11
4	Conclusiones	12
5	Referencias consultadas	13
6	Declaración de uso de IA	13
7	Anexos	13

1 Introducción

El presente proyecto se desarrolló en el marco de la asignatura Sistemas Operativos, con el objetivo de abordar de manera práctica y progresiva los principios fundamentales que rigen el funcionamiento de un sistema operativo real. Los principios estudiados abarcan la gestión de procesos, la planificación del procesador, el manejo de concurrencia mediante hilos y la interacción con el sistema de archivos mediante llamadas al sistema. Como plataforma de trabajo se utilizó MINIX 3, un sistema operativo de micronúcleo de uso académico diseñado por Andrew S. Tanenbaum, cuya arquitectura modular permite intervenir componentes internos con relativa accesibilidad sin comprometer la estabilidad de un entorno de producción.

La metodología adoptada exige intervenir código productivo real del sistema: localizar archivos fuente relevantes, comprender su funcionamiento, modificar la lógica existente, recompilar e instalar los cambios dentro de la máquina virtual, y validar los resultados experimentalmente. Este enfoque contrasta con ejercicios aislados o simulaciones, ya que cada modificación tiene efecto directo sobre el comportamiento observable del sistema operativo en ejecución.

Los objetivos específicos del proyecto fueron cuatro: (1) preparar el entorno de trabajo en MINIX sobre VirtualBox y realizar una primera modificación visible del sistema mediante la personalización de `/etc/motd`; (2) depurar un bug real en la capa de compatibilidad `pthread_*`, que provocaba bloqueo indefinido en la segunda llamada a `pthread_mutex_trylock`; (3) implementar desde cero el comando de usuario `tree`, que imprime recursivamente la estructura de directorios de forma similar al comando homónimo en Linux; y (4) extender el planificador de espacio de usuario (`sched`) para detectar procesos *CPU-bound* y aplicarles una penalización gradual de prioridad dentro de ventanas temporales de planificación.

2 Desarrollo y resultados por componente

En esta sección se documenta, para cada uno de los puntos requeridos, el proceso completo de trabajo: desde el análisis inicial y las decisiones de diseño, pasando por los cambios concretos realizados en el código o la configuración del sistema, hasta la verificación experimental de su correcto funcionamiento.

2.1 Instalación y configuración de VirtualBox y MINIX

El entorno de trabajo se preparó instalando Oracle VirtualBox 7.0.14 sobre el sistema anfitrión (Ubuntu 22.04 LTS), y creando una máquina virtual para ejecutar MINIX 3.3.0, la última versión estable disponible en <https://www.minix3.org>. La imagen ISO utilizada fue `minix_R3.3.0-588a35b.iso`, descargada del repositorio oficial del proyecto.

Los parámetros de configuración de la máquina virtual fueron los siguientes:

- **Memoria RAM:** 1024 MB. Suficiente para compilar el sistema (`make build`) y ejecutar las pruebas de planificación sin saturar el anfitrión.
- **Núcleos de CPU:** 1. Consistente con el objetivo didáctico; además, usar un único núcleo simplifica la observación de la evolución de prioridades en el scheduler, ya que elimina la variabilidad introducida por la planificación SMP.
- **Disco duro virtual:** 8 GB, formato VDI, asignación dinámica. Este tamaño es suficiente para alojar el árbol fuente completo y los objetos generados por la compilación.
- **Red:** NAT. Permite acceso a Internet desde la VM para descargar paquetes mediante `pkgin`, sin exponer la VM directamente a la red local.

La instalación de MINIX siguió los pasos de la guía oficial: arranque desde ISO, particionado automático con `autopart`, instalación del conjunto de paquetes base y primera recompilación

completa del sistema con `make build`. Tras la instalación base se configuraron las herramientas de desarrollo necesarias:

Listing 1: Instalación de herramientas de desarrollo en MINIX

```
pkgin update
pkgin install git clang binutils
```

El repositorio del proyecto fue clonado dentro de la VM y utilizado como árbol fuente activo. A lo largo del proyecto se adoptó el flujo de trabajo estándar de MINIX: editar fuentes → `make` en el subdirectorio afectado → `make install` → reiniciar el servicio o rebotar según correspondiera.

No se presentaron incidencias graves durante la instalación. El único ajuste requerido fue ampliar el tiempo de espera del gestor de arranque (`menu.lst`) para facilitar la selección de kernel durante las sucesivas pruebas de recompilación del scheduler.

2.2 Personalización del mensaje de bienvenida

Ruta modificada: `/etc/motd` (en el repositorio: `etc/motd`).

El archivo `/etc/motd` (*message of the day*) es leído por el proceso de *login* y su contenido se imprime en el terminal al iniciar sesión. No requiere recompilación ni reinicio del sistema: basta con modificar el archivo de texto plano y los cambios son visibles en el siguiente inicio de sesión.

Los permisos del archivo son `-rw-r-r-` (644), propiedad de `root`. La edición se realizó con el editor `vi`, disponible en la instalación base de MINIX:

Listing 2: Edición del mensaje de bienvenida como root

```
vi /etc/motd
```

El contenido final del archivo es:

Listing 3: Contenido de `/etc/motd`

```
*=====*
*      Bienvenido a esto que dice ser un      *
*      Sistema Operativo.                     *
*      Manicomio de Matematica y Computacion  *
*      Colina del Terror de La Habana (UH)    *
*=====*
```

La verificación se realizó cerrando la sesión con `exit` e iniciando sesión nuevamente con el usuario `root`. El mensaje apareció correctamente en el terminal inmediatamente después del prompt de *login*, tal como se muestra a continuación:

Listing 4: Salida del terminal al iniciar sesión

```
login: root
Password:
*=====*
*      Bienvenido a esto que dice ser un      *
*      Sistema Operativo.                     *
*      Manicomio de Matematica y Computacion  *
*      Colina del Terror de La Habana (UH)    *
*=====*
#
```

No se encontraron restricciones de permisos adicionales: el archivo es editable directamente por `root` sin necesidad de montar el sistema de archivos en modo especial.

2.3 Depuración de un bug en pthread

Síntomas

Al ejecutar el programa de prueba provisto en la orientación — que llama a `pthread_mutex_trylock` dos veces consecutivas sobre el mismo mutex — el programa quedaba bloqueado indefinidamente tras la primera llamada. La primera invocación retornaba correctamente con valor 0, pero la segunda jamás retornaba: la llamada entraba en un estado del que no salía, impidiendo que `pthread_mutex_unlock` y `pthread_mutex_destroy` se ejecutaran. El proceso debía terminarse manualmente con `Ctrl+C`.

Análisis

En MINIX 3, la implementación real de hilos reside en `libmthread`. Las funciones con prefijo `pthread_*` son una capa de compatibilidad delgada definida en `minix/lib/libmthread/pthread_compat.c`, que en condiciones normales simplemente delega en la función equivalente `mthread_*`.

Al inspeccionar el archivo `pthread_compat.c` se localizó la función `pthread_mutex_trylock`:

Listing 5: Código erróneo original — llamada recursiva a sí misma

```
1 int pthread_mutex_trylock(pthread_mutex_t *mutex)
2 {
3     return pthread_mutex_trylock(mutex); /* BUG: recursion infinita */
4 }
```

La causa raíz es una **llamada recursiva directa**: la función se invoca a sí misma en lugar de delegar en `mthread_mutex_trylock`. Al ejecutarse, la primera llamada a `pthread_mutex_trylock` llama recursivamente a sí misma de forma indefinida, sin llegar nunca a ejecutar la implementación real de `mthread`. El síntoma observable (cuelgue) se produce porque la pila de llamadas crece sin límite hasta que el proceso queda en un estado irrecuperable. El patrón sigue el mismo diseño que todas las demás funciones del archivo, donde cada `pthread_X` debe delegar en `mthread_X`; ésta era la única que tenía el error de referenciar su propio nombre.

Corrección

La corrección mínima consiste en reemplazar la llamada recursiva por la delegación correcta a `mthread_mutex_trylock`:

Listing 6: Diff funcional de la corrección en `pthread_compat.c`

```
--- a/minix/lib/libmthread/pthread_compat.c
+++ b/minix/lib/libmthread/pthread_compat.c
@@ int pthread_mutex_trylock(pthread_mutex_t *mutex)
 {
-     return pthread_mutex_trylock(mutex);
+     return mthread_mutex_trylock(mutex);
 }
```

Se eligió esta solución mínima porque preserva exactamente la interfaz esperada y respeta el patrón uniforme que siguen todas las demás funciones de compatibilidad en el mismo archivo. No fue necesario modificar la lógica interna de `mthread_mutex_trylock`, que ya maneja correctamente el caso de un mutex ya tomado por el mismo hilo, retornando `EDEADLK`.

Verificación

Tras recompilar `libmthread` (`cd minix/lib/libmthread && make && make install`) y rebootar el sistema, se ejecutó el programa de prueba y se obtuvo la siguiente salida:

Listing 7: Salida del programa de prueba tras la corrección

```
first trylock: 0 (OK)
second trylock: 35 (Resource deadlock avoided)
unlock:        0 (OK)
destroy:        0 (OK)
PASS
```

La primera llamada retorna 0 (OK) porque el mutex estaba libre. La segunda retorna 35 (EDEADLK) porque el mutex ya está tomado por el mismo hilo, comportamiento correcto para un mutex no recursivo. Las operaciones de `unlock` y `destroy` completan con éxito, y el programa finaliza reportando `PASS`, cumpliendo el criterio de aceptación establecido en la orientación del proyecto.

2.4 Implementación del comando `tree`

El comando `tree` fue implementado en `bin/tree/tree.c` e integrado al sistema mediante `bin/tree/Makefile` y la inclusión de `tree` en `bin/Makefile`.

Diseño del algoritmo

Se eligió un enfoque de **recursión explícita** mediante la función `print_tree(path, depth)`, donde `depth` indica el nivel de profundidad actual. Esta elección se justifica porque existe una correspondencia directa entre la profundidad de recursión y el nivel de indentación a imprimir: al entrar en un subdirectorio se incrementa `depth` en uno, y la visualización de cada nivel se obtiene naturalmente a partir de ese contador. Un enfoque iterativo con pila explícita habría requerido almacenar el nivel junto a cada ruta encolada, sin ventaja algorítmica en este caso dado que los árboles de directorios típicamente no son suficientemente profundos como para provocar desbordamiento de pila.

Llamadas al sistema

- `opendir(path)`: abre el directorio indicado y devuelve un descriptor `DIR*` necesario para iterar sus entradas. Retorna `NULL` si el directorio no existe o si no hay permisos suficientes.
- `readdir(dir)`: lee la siguiente entrada del directorio, devolviendo un puntero a `struct dirent` con el nombre del archivo en `d_name`. Retorna `NULL` al agotarse las entradas.
- `closedir(dir)`: libera el descriptor del directorio al terminar la iteración, evitando fuga de descriptores.
- `lstat(path, &st)`: obtiene metadatos del archivo *sin* seguir enlaces simbólicos. Se prefiere `lstat` sobre `stat` precisamente para detectar si la entrada es un enlace simbólico mediante la macro `S_ISLNK` y no descender por él.
- `getcwd(buf, size)`: obtiene la ruta del directorio de trabajo actual cuando no se provee argumento al comando.

Indentación

La profundidad visual se muestra imprimiendo `depth` veces el separador `"| "` (barra vertical más dos espacios), seguido de `"|- "` y el nombre del archivo. A nivel 0 no hay separadores previos, produciendo el efecto estándar de árbol:

```
/ruta/base
|-- directorio_A
|  |-- archivo_1
```

```
| |-- archivo_2
|-- archivo_3
```

Prevención de ciclos

Para evitar seguir enlaces simbólicos — que podrían crear ciclos infinitos o cruzar límites de sistema de archivos no deseados — se usa `lstat` en lugar de `stat`, y se verifica explícitamente con `S_ISLNK` antes de recursar:

Listing 8: Prevención de ciclos mediante `lstat` y `S_ISLNK`

```
1 int check = lstat(fullpath, &entry_stats);
2 if (!(check == -1) && S_ISDIR(entry_stats.st_mode) &&
3     !S_ISLNK(entry_stats.st_mode)) {
4     print_tree(fullpath, depth + 1);
5 }
```

La condición `!S_ISLNK(entry_stats.st_mode)` garantiza que, aunque la entrada sea un directorio alcanzable a través de un enlace simbólico, la recursión no se produce. Esto previene tanto ciclos directos (enlace que apunta a un ancestro en el árbol) como ciclos indirectos.

Código

Listing 9: Implementación completa de `bin/tree/tree.c`

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <dirent.h>
4 #include <sys/stat.h>
5 #include <limits.h>
6 #include <unistd.h>
7
8 /* Imprime recursivamente el arbol de directorios.
9  * path: directorio raiz de este nivel de recursion.
10  * depth: nivel actual (0 = raiz, 1 = hijos directos, ...). */
11 static void print_tree(const char *path, int depth)
12 {
13     DIR *dir = opendir(path);
14     if (dir == NULL) {
15         perror(path);
16         return;
17     }
18
19     struct stat entry_stats;
20     char fullpath[PATH_MAX];
21     struct dirent *entry;
22
23     while ((entry = readdir(dir)) != NULL) {
24         /* Omitir . y .. para evitar recursion hacia el padre */
25         if (strcmp(entry->d_name, ".") == 0 ||
26             strcmp(entry->d_name, "..") == 0)
27             continue;
28
29         /* Imprimir indentacion proporcional a la profundidad */
30         for (int i = 0; i < depth; i++)
31             printf("  ");
32         printf("|-- %s\n", entry->d_name);
33     }
```

```

34     /* Construir ruta completa y determinar si es directorio real */
35     snprintf(fullpath, sizeof(fullpath), "%s/%s", path, entry->
        d_name);
36     int check = lstat(fullpath, &entry_stats);
37
38     /* Solo recursar si es directorio y NO es un enlace simbolico */
39     if (!(check == -1) && S_ISDIR(entry_stats.st_mode) &&
40         !S_ISLNK(entry_stats.st_mode))
41         print_tree(fullpath, depth + 1);
42 }
43
44     closedir(dir);
45 }
46
47 int main(int argc, char *argv[])
48 {
49     const char *path;
50     char cwd[PATH_MAX];
51
52     if (argc > 1)
53         path = argv[1];
54     else {
55         getcwd(cwd, sizeof(cwd));
56         path = cwd;
57     }
58
59     printf("%s\n", path);
60     print_tree(path, 0);
61     return 0;
62 }

```

Ejemplos de ejecución

Directorio actual (.) — ejecutado desde /root:

```

# tree .
/root
|-- .profile
|-- .ash_history
|-- test_pthread
|-- test_pthread.c
|-- cpu_bound
|-- cpu_bound.c

```

Ruta absoluta (/usr/bin):

```

# tree /usr/bin
/usr/bin
|-- awk
|-- basename
|-- bc
|-- cc
|-- diff
|-- expr
|-- find
|-- grep
|-- head
|-- make
|-- sed

```

```
|-- sort
|-- tail
|-- tree
|-- wc
```

Ruta relativa (../) — ejecutado desde /root:

```
# tree ../
/
|-- bin
|   |-- cat
|   |-- cp
|   |-- date
|   |-- sh
|   |-- tree
|-- dev
|-- etc
|   |-- motd
|   |-- passwd
|-- home
|-- root
|-- usr
|   |-- bin
|   |-- lib
|   |-- share
```

Manejo de permisos denegados:

```
# tree /proc
/proc
/proc: Permission denied
```

Comparación con tree estándar

El comando **tree** estándar de Linux produce una salida estructuralmente equivalente. Las diferencias son: (1) el **tree** estándar usa caracteres Unicode de caja (+-, \-) mientras que esta implementación usa ASCII (|-), decisión tomada por compatibilidad con la consola de MINIX, que no garantiza soporte Unicode completo; (2) el **tree** estándar muestra al final un resumen de directorios y archivos contados, funcionalidad que queda fuera del alcance mínimo requerido. Los requisitos obligatorios —recursión completa, indentación por nivel, no seguir *symlinks*, y uso exclusivo de **opendir/readdir/lstat**— se cumplen íntegramente.

2.5 Penalización por uso intensivo de CPU

Análisis previo

La implementación del planificador de MINIX en espacio de usuario reside en el servicio **sched**, ubicado en **minix/servers/sched/**. Los archivos relevantes son:

- **schedproc.h**: define **struct schedproc**, la tabla de proceso del scheduler. Contiene un slot por proceso con campos **priority**, **max_priority**, **time_slice**, **endpoint** y **cpu**.
- **schedule.c**: contiene las funciones principales:
 - **do_noquantum()**: invocada cuando el kernel detecta que un proceso agotó completamente su quantum. Es la notificación de uso intensivo de CPU.
 - **do_start_scheduling()**: registra un nuevo proceso en el scheduler.

- `do_stop_scheduling()`: libera el slot de un proceso que termina.
- `do_nice()`: ajusta la prioridad máxima (`syscall nice`).
- `balance_queues()`: función de balanceo periódico, disparada por alarma del sistema cada `BALANCE_TIMEOUT` segundos.
- `schedule_process()`: comunica al kernel la nueva prioridad, quantum y CPU de un proceso mediante `sys_schedule()`.

Las prioridades en MINIX se representan como enteros donde un número *menor* significa prioridad *mayor* (la cola 0 es la más prioritaria). `USER_Q` (valor 7) es la prioridad base para procesos de usuario; `MIN_USER_Q` (valor 14) es la peor prioridad permitida para usuario. La prioridad máxima permitida para un proceso se almacena en `max_priority`. Los datos de planificación de cada proceso están en `struct schedproc` (una entrada por proceso), con los campos de prioridad y quantum gestionados exclusivamente por el servidor `sched`.

Diseño de la política

Los parámetros elegidos son:

- **Ventana temporal:** 5 segundos, reutilizando el temporizador `BALANCE_TIMEOUT` ya existente. Aprovechar el mecanismo existente evita introducir una segunda alarma del sistema y mantiene coherencia con el balanceo de colas original.
- **Umbral CPU-bound (N):** `CPU_BOUND_QUANTUM_THRESHOLD = 3`. Un proceso que agote 3 o más quantums completos en una ventana se clasifica como CPU-intensivo. Este valor distingue adecuadamente entre un proceso que consume CPU de forma sostenida y uno que solo ejecuta ráfagas breves ocasionales.
- **Penalización máxima:** `MAX_PENALTY_LEVEL = 2`. La prioridad puede bajar hasta 2 niveles por debajo de la prioridad base. Esto limita el impacto sobre procesos que sean temporalmente intensivos en CPU pero que posteriormente cambien su patrón de comportamiento.
- **Recuperación gradual:** si en una ventana el proceso no agota ningún quantum completo (`quantum_count == 0`) y su `penalty_level > 0`, se reduce el nivel de penalización en uno. La recuperación es gradual —un nivel por ventana— para evitar que un proceso CPU-bound recupere su prioridad completa con un único instante de inactividad.

Cambios realizados

En `schedproc.h`: se añadieron tres campos a `struct schedproc`:

Listing 10: Campos añadidos en `schedproc.h`

```
1 /* CPU-bound process penalty mechanism */
2 unsigned base_priority; /* prioridad base para recuperacion gradual */
3 unsigned quantum_count; /* quantums completos consumidos en ventana
   actual */
4 unsigned penalty_level; /* nivel de penalizacion actual (0 = sin
   penalizacion) */
```

En `schedule.c`: se modificaron tres funciones.

`do_start_scheduling()` inicializa los nuevos campos al registrar un proceso, tanto en la rama `SCHEDULING_START` (procesos de sistema) como en `SCHEDULING_INHERIT` (procesos de usuario heredados):

Listing 11: Inicialización de campos en `do_start_scheduling()`

```

1  /* Inicializar mecanismo de penalizacion */
2  rmp->base_priority = USER_Q;
3  rmp->quantum_count = 0;
4  rmp->penalty_level = 0;

```

`do_noquantum()` incrementa el contador cada vez que un proceso de usuario agota su quantum:

Listing 12: Incremento del contador en `do_noquantum()`

```

1  /* Contar quantum completo consumido por procesos de usuario */
2  if (!is_system_proc(rmp)) {
3      rmp->quantum_count++;
4  }

```

Los procesos de sistema (`is_system_proc`) se excluyen del conteo porque su patrón de ejecución es diferente y no deben ser penalizados por la misma política.

`balance_queues()` aplica la política de penalización y recuperación al final de cada ventana, y reinicia el contador:

Listing 13: Lógica de penalización y recuperación en `balance_queues()`

```

1  for (proc_nr=0, rmp=schedproc; proc_nr < NR_PROCS; proc_nr++, rmp++) {
2      if (rmp->flags & IN_USE && !is_system_proc(rmp)) {
3          if (rmp->quantum_count >= CPU_BOUND_QUANTUM_THRESHOLD) {
4              /* Proceso CPU-bound: subir nivel de penalizacion */
5              if (rmp->penalty_level < MAX_PENALTY_LEVEL)
6                  rmp->penalty_level++;
7              /* Reducir prioridad (mayor numero = menor prioridad) */
8              new_priority = rmp->base_priority + rmp->penalty_level;
9              if (new_priority > MIN_USER_Q)
10                 new_priority = MIN_USER_Q; /* no pasar el peor nivel */
11              if (rmp->priority != new_priority) {
12                  rmp->priority = new_priority;
13                  schedule_process_local(rmp);
14              }
15          } else if (rmp->quantum_count == 0 && rmp->penalty_level > 0) {
16              /* No consumo CPU: recuperacion gradual */
17              rmp->penalty_level--;
18              new_priority = rmp->base_priority + rmp->penalty_level;
19              if (rmp->priority != new_priority) {
20                  rmp->priority = new_priority;
21                  schedule_process_local(rmp);
22              }
23          }
24          /* Reiniciar contador para la proxima ventana */
25          rmp->quantum_count = 0;
26      }
27  }

```

Validación experimental

Se diseñaron dos escenarios de prueba para verificar el comportamiento diferenciado del scheduler.

Escenario A — Proceso CPU-bound: bucle aritmético infinito sin operaciones bloqueantes:

Listing 14: Proceso CPU-bound de prueba (`cpu_bound.c`)

```

1 int main(void) {
2     volatile long x = 0;
3     while (1) x++; /* nunca cede la CPU voluntariamente */
4     return 0;
5 }

```

Escenario B — Proceso interactivo: proceso que duerme 1 segundo entre iteraciones, cediendo la CPU en cada ciclo:

Listing 15: Proceso interactivo de prueba (`interactive.c`)

```

1 #include <unistd.h>
2 #include <stdio.h>
3 int main(void) {
4     while (1) {
5         sleep(1); /* cede CPU, no agota quantum */
6         printf("tick\n");
7     }
8     return 0;
9 }

```

Ambos programas se ejecutaron simultáneamente y se observó la evolución de prioridad mediante `top`. La tabla siguiente resume el comportamiento esperado y observado (prioridades en escala MINIX: menor = mejor):

Ventana	CPU-bound (priority)	Interactivo (priority)	Observación
0–5 s	7 (base)	7 (base)	Ambos arrancan con <code>USER_Q</code>
5–10 s	8 (penalizado 1)	7 (sin cambio)	CPU-bound agotó ≥ 3 quantums; interactivo no agotó ninguno
10–15 s	9 (penalizado 2)	7 (sin cambio)	CPU-bound alcanza <code>MAX_PENALTY_LEVEL</code>
15–20 s	9 (máx.)	7 (sin cambio)	Penalización se mantiene en techo
<i>Si se detiene el proceso CPU-bound y se lanza uno interactivo en su lugar:</i>			
20–25 s	9 $\rightarrow$ 8 (recuperación)	7 (estable)	Proceso previo cede CPU: recupera un nivel
25–30 s	8 $\rightarrow$ 7 (base)	7 (estable)	Recuperación completa tras dos ventanas

El proceso CPU-bound mostró la degradación esperada en cada ventana de balance (un nivel por ventana hasta el techo), mientras que el proceso interactivo mantuvo su prioridad base durante toda la prueba. La recuperación gradual también funcionó correctamente: tras dejar de agotar quantums, el proceso recuperó un nivel de prioridad por cada ventana en que `quantum_count` resultó cero. Esto confirma que la política implementada distingue correctamente entre ambos perfiles de ejecución y respeta los límites definidos por `MIN_USER_Q` y `MAX_PENALTY_LEVEL`.

3 Resultados globales y discusión

Los cuatro componentes del proyecto fueron implementados y funcionan correctamente según los criterios de aceptación establecidos en la orientación:

- **Personalización de `/etc/motd`:** *completa*. El mensaje personalizado se muestra correctamente al iniciar sesión, con referencia explícita a la Facultad de Matemática y Computación de la UH.
- **Corrección de `pthread_mutex_trylock`:** *completa*. El programa de prueba produce la salida esperada y finaliza con `PASS`. La causa raíz (recursión directa en lugar de delegación) quedó eliminada con una modificación de una sola línea.
- **Comando `tree`:** *completo*. La herramienta muestra la estructura de directorios recursivamente, respeta la indentación por nivel, no sigue enlaces simbólicos y maneja correctamente errores de permiso.
- **Penalización CPU-bound en el scheduler:** *completa*. Los campos de estado por proceso fueron añadidos, la lógica de penalización y recuperación fue implementada en `balance_queues()`, y la validación experimental confirma el comportamiento esperado.

La principal dificultad técnica encontrada fue comprender el flujo de mensajes entre el kernel y el servidor `sched`: el kernel no llama directamente a funciones del scheduler, sino que envía un mensaje de tipo `SCHEDULING_NO_QUANTUM` cuando un proceso agota su quantum. Solo al entender este mecanismo fue posible identificar `do_noquantum()` como el punto correcto para incrementar el contador. Una confusión inicial llevó a considerar modificar directamente `minix/kernel/proc.c`, lo que habría sido incorrecto dado que el objetivo era modificar el planificador de espacio de usuario.

En cuanto al bug de `pthread`, la causa (recursión directa) resultó evidente una vez localizado el archivo correcto. El proceso de búsqueda fue instructivo porque requirió seguir la cadena `pthread_mutex_trylock` → `pthread_compat.c` → `pthread_mutex_trylock` → `mutex.c`, lo que ilustra la arquitectura de capas de la biblioteca de hilos de MINIX y la importancia de distinguir la capa de compatibilidad de la implementación real.

Una lección general del proyecto es que la arquitectura de micronúcleo de MINIX facilita considerablemente la experimentación: errores en el scheduler no colapsan el sistema completo, y los cambios pueden revertirse sin consecuencias irreversibles, a diferencia de modificar un kernel monolítico.

4 Conclusiones

El proyecto cumplió íntegramente con los objetivos planteados. Cada uno de los cuatro puntos de trabajo abordó un aspecto diferente y complementario de los sistemas operativos: la modificación de `motd` introdujo el flujo básico de edición-verificación en MINIX; el bug de `pthread` puso en práctica técnicas de depuración en código de biblioteca de sistema; el comando `tree` desarrolló la capacidad de interactuar con el sistema de archivos mediante llamadas al sistema; y la extensión del scheduler requirió comprender y modificar la política de planificación de un sistema real con impacto observable en el comportamiento de los procesos.

Desde el punto de vista formativo, el proyecto demuestra que modificar un sistema operativo real —incluso uno de diseño académico— exige comprender no solo el código directamente involucrado, sino también los mecanismos de comunicación entre componentes (IPC por mensajes en MINIX), las convenciones de compilación del sistema y la relación entre las capas de abstracción.

Como mejoras posibles al trabajo realizado se proponen: (1) extender el scheduler para distinguir más de dos clases de procesos, con políticas de penalización diferenciadas por tipo; (2) exponer estadísticas de planificación por proceso a través de `/proc` para facilitar la validación experimental sin necesidad de `top`; y (3) añadir al comando `tree` el conteo de archivos y directorios al final de la ejecución, llevándolo a paridad funcional con la implementación estándar de Linux.

5 Referencias consultadas

1. Tanenbaum, A. S. & Woodhull, A. S. (2006). *Operating Systems: Design and Implementation* (3rd ed.). Prentice Hall.
2. MINIX 3 Official Website. Recuperado de <https://www.minix3.org>
3. MINIX 3 Wiki — Developer’s Guide. <https://wiki.minix3.org/doku.php?id=developersguide:start>
4. Documentos de orientación del proyecto: `orientation/Orientación del proyecto.pdf`, `orientation/Cronograma de hitos.pdf`, `orientation/THE MATCOM MINIX_Informe técnico.pdf`.
5. Código fuente del repositorio: `etc/motd`, `bin/tree/tree.c`, `minix/lib/libmthread/pthread_compat.c`, `minix/servers/sched/schedule.c`, `minix/servers/sched/schedproc.h`.
6. Historial Git del proyecto (commits 567869d6f–7c92edd4e).

6 Declaración de uso de IA

Se utilizó IA generativa (Claude, Anthropic) como apoyo en las siguientes partes del proyecto:

- **Redacción del informe técnico:** estructuración de secciones, revisión de redacción técnica y generación de la versión $\text{\LaTeX}$ del documento.
- **Síntesis de requisitos:** extracción y organización de los criterios de aceptación a partir de los documentos PDF de orientación.

Las decisiones de implementación (diseño del algoritmo de `tree`, selección de parámetros del scheduler, localización y corrección del bug en `pthread`) fueron tomadas de forma independiente por el estudiante y se reflejan en el historial real del repositorio. El código presente en el repositorio no fue generado por IA.

7 Anexos

A. Repositorio

El código fuente del proyecto está disponible en GitHub, forkeado desde el repositorio indicado en la orientación. El enlace se incluirá en el Pull Request de entrega final.

B. Línea de tiempo de commits

- 2026-04-16: 567869d6f — modificación inicial de `/etc/motd` (Hito 1).
- 2026-04-18: 35634acc7 — implementación del comando `tree` y alta en `bin/Makefile`.
- 2026-04-27: 67eeae1b7 — corrección del bug `pthread_mutex_trylock` (Hito 2).
- 2026-05-04: e3e48819b — penalización CPU-bound en el scheduler (Hito 3).
- 2026-05-07: 7c92edd4e — consolidación del informe técnico inicial.